

Практическое занятие №6

Технологии индустриального программирования

Дисциплина: Технологии индустриального программирования

Институт: ИСТИП

Кафедра: Индустриального программирования

Преподаватель: Астафьев Рустам Уралович

Студент: Петрушин Евгений

Семестр: 1 семестр, 2026/2027 уч. год

Задание №1

Условие:

Инфлюэнсер Аркадий выбирает число N (от 1 до 100), выбирает стену с квадратом из кирпичей $N \times N$ и заполняет его по правилу:

1. Все кирпичи выше побочной диагонали — цифрой 0.
2. Все кирпичи на побочной диагонали — цифрой 1.
3. Все кирпичи ниже побочной диагонали — цифрой 2.

Необходимо спрогнозировать результат его работы.

Пример:

Ввод: 4

Вывод:

```
0 0 0 1
0 0 1 2
0 1 2 2
1 2 2 2
```

Решение (код программы на Python):

```
n = int(input())

for i in range(n):
    row = []
    for j in range(n):
        # Побочная диагональ: i + j == n - 1
        if i + j < n - 1:
            row.append('0') # выше побочной диагонали
        elif i + j == n - 1:
            row.append('1') # на побочной диагонали
        else:
            row.append('2') # ниже побочной диагонали
    print(' '.join(row))
```

Пояснение:

Побочная диагональ квадратной матрицы $N \times N$ — это элементы, для которых $i + j = N - 1$ (где i — номер строки, j — номер столбца, нумерация с 0). Элементы выше побочной диагонали удовлетворяют условию $i + j < N - 1$, ниже — $i + j > N - 1$.

Проверка для $N = 4$:

```
i=0: j=0→0+0=0<3→0; j=1→0+1=1<3→0; j=2→0+2=2<3→0; j=3→0+3=3=3→1 ⇒ 0 0 0 1
i=1: j=0→1<3→0; j=1→1+1=2<3→0; j=2→1+2=3=3→1; j=3→1+3=4>3→2 ⇒ 0 0 1 2
i=2: j=0→2<3→0; j=1→2+1=3=3→1; j=2→2+2=4>3→2; j=3→2+3=5>3→2 ⇒ 0 1 2 2
i=3: j=0→3=3=3→1; j=1→3+1=4>3→2; j=2→3+2=5>3→2; j=3→3+3=6>3→2 ⇒ 1 2 2 2
```

Результат совпадает с примером. ✓

Задание №2

Условие:

Кирилл очень любит грибы. Друг Александр присылает ему координаты грибов (ячейки сетки). Необходимо составить таблицу, в которой местоположение грибов обозначается *, а остальные ячейки содержат количество грибов вокруг данной точки (как в сапёре). Все символы выводятся без пробелов.

Ввод: N M — строки и столбцы; W — количество грибов; далее W пар координат.

Пример:

Ввод:

```
3 2
2
1 1
2 2
```

Вывод:

```
*2
2*
11
```

Решение (код программы на Python):

```
n, m = map(int, input().split())
w = int(input())

# Создаём поле NxM, заполненное нулями
field = [[0] * m for _ in range(n)]

# Считываем координаты грибов (1-индексация -> 0-индексация)
mushrooms = set()
for _ in range(w):
    r, c = map(int, input().split())
    r -= 1 # переводим в 0-индексацию
    c -= 1
    mushrooms.add((r, c))

# Для каждой клетки с грибом увеличиваем счётчики соседей
# 8 направлений: вверх, вниз, влево, вправо и 4 диагонали
directions = [(-1, -1), (-1, 0), (-1, 1),
              (0, -1),          (0, 1),
              (1, -1),  (1, 0),  (1, 1)]

for (r, c) in mushrooms:
    for dr, dc in directions:
        nr, nc = r + dr, c + dc
        if 0 <= nr < n and 0 <= nc < m:
            field[nr][nc] += 1

# Вывод результата
for i in range(n):
    row = ''
    for j in range(m):
        if (i, j) in mushrooms:
            row += '*'
        else:
            row += str(field[i][j])
    print(row)
```

Пояснение:

Алгоритм аналогичен игре «Сапёр»:

1. Создаётся поле NxM, заполненное нулями.

2. Для каждого гриба (координаты в 1-индексации, переводим в 0-индексацию) отмечаем его позицию.
3. Для каждого гриба увеличиваем на 1 счётчики всех соседних клеток (8 направлений).
4. При выводе: если клетка содержит гриб — печатаем *, иначе — количество грибов-соседей.

Проверка для примера (3x2, грибы: (1,1) и (2,2)):

Поле 3x2 (0-инд.): грибы в (0,0) и (1,1).

Гриб (0,0): соседи (0,1)+=1, (1,0)+=1, (1,1)+=1.

Гриб (1,1): соседи (0,0)+=1, (0,1)+=1, (1,0)+=1, (2,0)+=1, (2,1)+=1.

Итого: field[0][1]=2, field[1][0]=2, field[2][0]=1, field[2][1]=1.

Вывод: *2 / 2* / 11. Совпадает с примером. ✓

Задание №3

Условие:

Учитель математики написал последовательность, начинающуюся с числа X. Каждый следующий элемент описывает предыдущий (последовательность Look-and-say):

```
1
11      (одна 1)
21      (две 1)
1211    (одна 2, одна 1)
111221  (одна 1, одна 2, две 1)
312211  (три 1, две 2, одна 1)
...
```

Программа получает на вход число X (первый член) и число N (номер элемента). Необходимо вывести N-ую строку последовательности.

Пример:

Ввод: 1 4

Вывод: 1211

Решение (код программы на Python):

```
x, n = input().split()
# x - первый член (строка), n - номер элемента
n = int(n)

current = str(x)

for step in range(n - 1):
    # Генерируем следующий элемент по правилу Look-and-say
    result = ''
    i = 0
    while i < len(current):
        digit = current[i]
        count = 1
        # Считаем подряд идущие одинаковые символы
        while i + count < len(current) and current[i + count] == digit:
            count += 1
        result += str(count) + digit
        i += count
    current = result

print(current)
```

Пояснение:

Последовательность Look-and-say (читай-и-говори) строится так: каждый следующий элемент является «описанием» предыдущего. Мы читаем предыдущую строку слева направо, группируя подряд идущие одинаковые цифры, и для каждой группы записываем количество повторений и саму цифру.

Алгоритм:

1. Начинаем с `current = str(x)` (первый элемент).
2. Повторяем $(n-1)$ раз:
 - a. Проходим по `current`, находя группы одинаковых символов.
 - b. Для каждой группы записываем: количество + символ.
 - c. Результат становится новым `current`.
3. Выводим итоговый `current`.

Проверка (x=1, n=4):

Шаг 1: `current = '1'`

Шаг 2: `'1' → одна 1 → '11'. current = '11'`

Шаг 3: `'11' → две 1 → '21'. current = '21'`

Шаг 4: `'21' → одна 2, одна 1 → '1211'. current = '1211'`

Итог (n=4): `'1211'`. Совпадает с примером. ✓

Дополнительная проверка (x=1, n=6):

n=5: `'1211' → одна 1, одна 2, две 1 → '111221'. ✓`

n=6: `'111221' → три 1, две 2, одна 1 → '312211'. ✓`